

Rapport Projet C++

I - Introduction :

Problématique : Intrusion de cambrioleurs dans un bâtiment.

Dans le cadre du Bureau d'Étude du projet de C++, nous avons cherché à répondre à une problématique qui touche beaucoup de personnes : la sécurisation d'un bâtiment.

L'objectif de notre projet est d'utiliser les différents périphériques mis à notre disposition ainsi que les fonctions de la carte nodeMCU pour réaliser un système de sécurité opérationnel qui permet de détecter une intrusion dans un bâtiment.

Étant donné que l'objectif du BE est de pratiquer l'utilisation du C++ ainsi que la programmation/conception orientée objet, nous avons décomposé notre projet en différentes classes en utilisant les mécanismes d'héritage, l'utilisation d'exceptions, la redéfinition d'opérateurs et l'utilisation de la STL.

Le système repose sur plusieurs piliers fonctionnels :

- **Détection et surveillance** : Utilisation de capteurs ultrasoniques pour surveiller l'état physique d'une porte et détecter des mouvements suspects.
- **Action et dissuasion** : Pilotage d'un servomoteur pour le verrouillage (Loquet) et déclenchement d'alertes sonores et visuelles (Buzzer et LEDs).
- **Connectivité IoT** : Mise en place d'un serveur web asynchrone pour notifier l'utilisateur en temps réel et permettre le pilotage du loquet de la porte à distance via Wi-Fi.
- **Fiabilité logicielle** : Utilisation des mécanismes avancés du C++ (STL pour le filtrage, gestion des exceptions pour les pannes matérielles) afin de garantir la résilience du système.

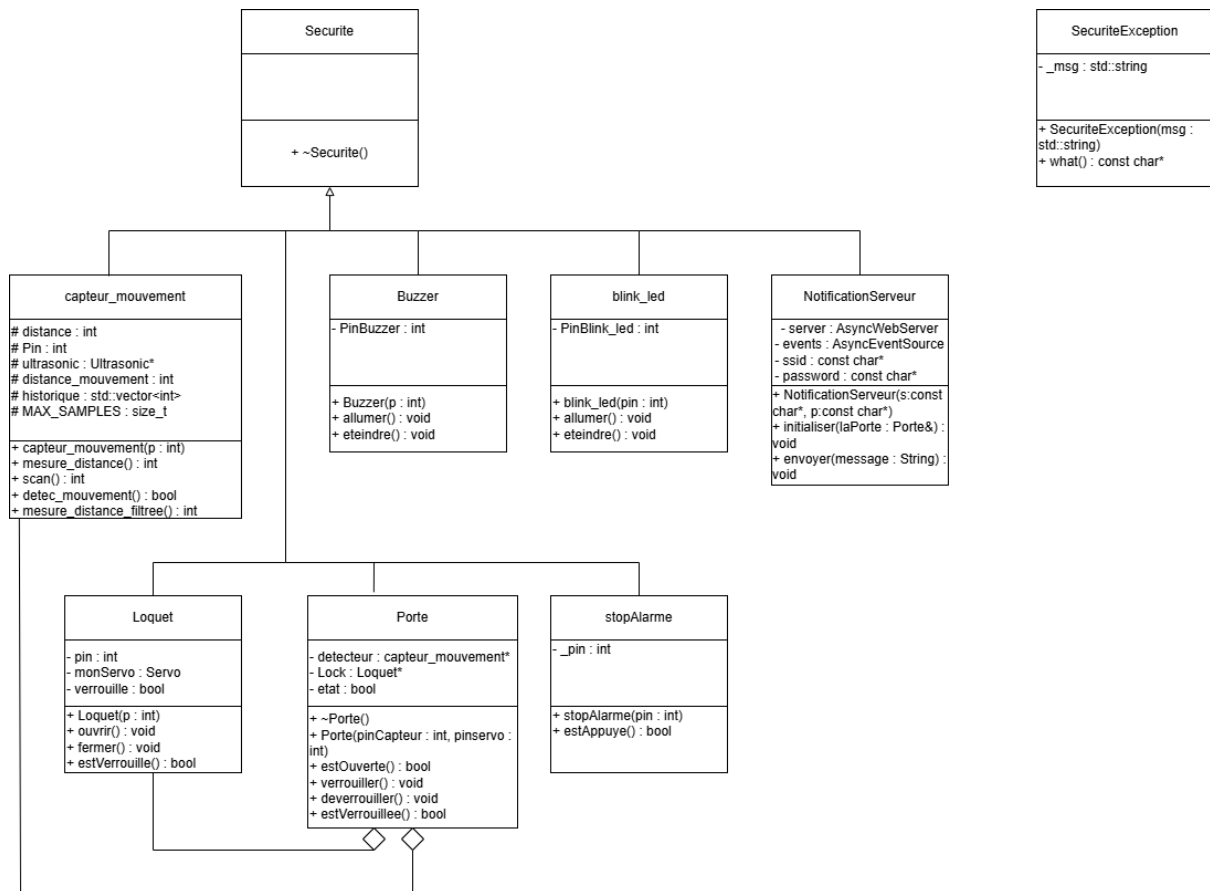
II - Conception orientée objet :

A / Hiérarchie de classes et Héritage

Le système repose sur une classe mère Sécurité. Toutes les composantes matérielles et logicielles héritent de cette base, ce qui permet d'uniformiser l'interface du système :

- **Capteurs et Actionneurs** : Les classes capteur_mouvement, Loquet, Buzzer, blink_led et stopAlarme héritent directement de Securite.
- **Logique et communication** : Les classes Porte et NotificationServeur s'inscrivent également dans cette hiérarchie pour assurer la cohérence globale du projet.

B / Diagramme de classes



C / Diagrammes de séquence

1. Diagramme de séquence de la sécurité

Ce premier diagramme de séquence illustre l'intrusion d'un inconnu dans le bâtiment. Dans un premier temps, l'inconnu ouvre la porte, le capteur de la porte détecte plus d'un centimètre donc l'alarme se met à sonner et les leds à clignoter 20 fois et l'utilisateur reçoit une notification. Ensuite, si le propriétaire souhaite éteindre l'alarme, il doit composer un code (appuyer sur les 2 boutons simultanément).

```

sequenceDiagram
    actor Utilisateur as Utilisateur (Web)
    participant NS as :NotificationServeur
    participant P as :Porte
    participant L as :Loquet
    participant CM as :capteur_mouvement

    Note over Utilisateur: Requête de changement d'état
    Utilisateur->>NS: 
    NS->>P: estVerrouille()
    P->>L: estVerrouille()
    L->>NS: status: bool
    Note over L: 
    alt Cas 1: La porte est déverrouillée -> Verrouiller
        P->>L: estOuverte()
        L->>CM: mesureDistance()
        CM->>P: distance: int
        Note over CM: 
        alt Porte fermée
            P->>L: fermer()
            L->>L: write()
        else Porte ouverte
            P->>NS: throw SecuriteException
            Note over P: impossible de verrouiller
        end
    end
    NS-->>Utilisateur: Réponse "FERME"
    alt Cas 2: La porte est verrouillée -> Déverrouiller
        NS->>P: deverrouiller()
        P->>L: ouvrir()
        L->>L: write()
    end
    NS-->>Utilisateur: Réponse "OUVERT"
  
```

III - Mécanismes Avancés du C++

Pour répondre aux exigences de fiabilité logicielle, nous avons intégré plusieurs fonctionnalités avancées du langage :

- Gestion des Exceptions : Nous avons créé la classe `SecuriteException`. Elle est levée en cas de défaillance matérielle (ex: capteur débranché renvoyant 0 cm) ou de commande illogique (tenter de verrouiller le loquet alors que la porte est physiquement ouverte).
- Utilisation de la STL : La classe `capteur_mouvement` utilise `std::vector` pour stocker un historique des mesures. L'algorithme `std::accumulate` est employé pour calculer une moyenne glissante, filtrant ainsi le bruit numérique du capteur ultrason.
- Redéfinition d'Opérateur : L'opérateur `<<` a été surchargé pour la classe `Porte` afin de permettre un affichage rapide et formaté de son état complet (physique et verrouillage) dans le serial monitor.

IV - Conclusion

Ce projet nous a permis de mettre en pratique les piliers de la programmation orientée objet dans un contexte IoT réel. L'utilisation du C++ a rendu le code robuste face aux erreurs matérielles grâce aux exceptions et efficace dans le traitement des données grâce à la STL. Le but a été atteint : nous disposons d'un système fonctionnel, pilotable à distance, capable de distinguer une ouverture normale d'une intrusion suspecte. Une perspective d'évolution serait l'ajout d'un module de stockage (carte SD) pour journaliser les intrusions même en cas de perte de connexion Wi-Fi.

V - Logiciels utilisés

Conception Orientée Objet : [Draw.io](https://app.diagrams.net/) (<https://app.diagrams.net/>)

Développement du code : Arduino IDE (<https://www.arduino.cc/en/software/>)

Gestion de version : GitHub (https://github.com/Zidane4124/BE_POO_MILI_BENAMOR/tree/Melik/project_name)